



PHASE P2 — PAYOUTS AUTOMATISÉS

Guide de test sandbox Wise Business & GoCardless PIS

Configuration, credentials et scénarios end-to-end
pour les virements sortants automatisés

VERSION 1.0 · MAI 2026 · ROADMAP P2

Table des matières

01.	Vue d'ensemble & architecture	3
02.	Setup ngrok (tunnel local)	5
03.	Wise Business — Sandbox immédiat	6
04.	GoCardless PIS — Sandbox immédiat	9
05.	Configuration Clenzy	12
06.	Scénarios end-to-end	14
07.	Débogage & commandes utiles	17
08.	Erreurs courantes & solutions	19
09.	Annexes — webhooks & structure des payloads	20

1. Vue d'ensemble & architecture

Ce guide accompagne le déploiement local des deux nouveaux **rails de payouts sortants** automatisés de Clenzy : Wise Business pour les virements internationaux (Maroc, Arabie Saoudite, 80+ pays) et GoCardless Bank Account Data + PIS pour les virements SEPA automatiques sans upload manuel.

Les 5 méthodes de payout supportées par Clenzy

MÉTHODE	COUVERTURE	NIVEAU D'AUTO	FRAIS TYPIQUES
STRIPE_CONNECT	EU, US, UK, +40 pays	Auto complet	0.25% + €0.10
WISE	80+ pays (dont MA, KSA)	Auto complet	~0.5%
OPEN_BANKING	EU (PSD2)	Auto via PIS	~€0.10/virement
SEPA_TRANSFER	EU (zone SEPA)	Semi-auto (XML + upload manuel)	Variable banque
MANUAL	—	Hors-Clenzy	N/A

Architecture Strategy + Registry

Tous les exécuteurs implémentent l'interface {@code PayoutExecutor} et sont auto-découverts par Spring. L'orchestrateur {@code PayoutExecutionService} délègue à l'exécuteur via {@code PayoutExecutorRegistry.get(method)}.

```
com.clenzy.payment.payout/
├─ PayoutExecutor           (interface)
├─ PayoutExecutorRegistry   (auto-discovery Spring)
├─ PayoutNotifier           (notifications partagées)
├─ wise/WiseClient          (HTTP REST Wise API)
├─ openbanking/
│   ├── GoCardlessPisClient  (HTTP REST GoCardless + cache token)
│   └─ OpenBankingProvider   (enum GOCARDLESS / TINK)
└─ executor/
    ├── StripeConnectPayoutExecutor
    ├── SepaTransferPayoutExecutor
    ├── ManualPayoutExecutor
    ├── WisePayoutExecutor
    └─ OpenBankingPayoutExecutor
```

Prérequis machine

OUTIL	VERSION	VÉRIFICATION
Docker + Docker Compose	récent	<code>docker compose version</code>
Java JDK	21+	<code>java -version</code>
Node.js	20+	<code>node --version</code>
ngrok	3.x	<code>ngrok version</code> — voir section 2

État de la roadmap P2 (mai 2026)

PR	DESCRIPTION	STATUT	TESTS
PR8	Refactor Strategy + Wise Business	Livré	12 tests
PR9	Open Banking PIS (GoCardless)	Livré	9 tests
PR10	Frontend SCA + endpoints (init/callback)	Livré	TS clean
PR11	Doc sandbox (ce guide)	Livré	—

Suite de tests backend

3557 tests verts (TypeScript clean côté frontend). Tous les flows critiques sont couverts par des tests unitaires : signature Wise, cache token GoCardless, validation consent, expiration, refunds.

2. Setup ngrok (tunnel local)

Pourquoi ngrok ?

Wise et GoCardless envoient des notifications webhook depuis leurs serveurs externes vers Clenzy à chaque changement d'état de virement. Votre `localhost:8084` n'est pas joignable depuis Internet — ngrok crée un tunnel HTTPS sécurisé qui relaie les requêtes vers votre machine.

Installation

```
$ brew install ngrok/ngrok/ngrok
```

Authentification

- 1 Créer un compte gratuit sur `ngrok.com`
- 2 Copier le token depuis le dashboard ngrok (section *Your Authtoken*)
- 3 Configurer le token localement :

```
$ ngrok config add-authtoken <votre-token-ngrok>
```

Lancer le tunnel sur le port backend Clenzy

```
$ ngrok http 8084
```

Une URL publique est affichée :

```
Forwarding https://a1b2c3d4.ngrok-free.app -> http://localhost:8084
```

C'est cette URL que vous allez configurer dans Wise et dans GoCardless comme URL de webhook.

Vérification

```
$ curl https://a1b2c3d4.ngrok-free.app/api/webhooks/payouts/wise/transfers-state-change
```

Doit retourner `401 Missing signature` — c'est attendu : confirme que le tunnel route correctement.

URL ngrok éphémère

Sur le plan gratuit, l'URL ngrok change à chaque relance. Si vous redémarrez ngrok :

- Mettre à jour l'URL de webhook dans Wise (Dashboard → Settings → Webhooks)
- Mettre à jour l'URL dans GoCardless (Dashboard → Developers → Webhooks)
- Mettre à jour `gocardless.debtor-account-id` n'est pas concerné — il reste fixe

Le plan payant ngrok (\$8/mois) permet d'avoir un sous-domaine fixe ce qui simplifie nettement le workflow.

3. Wise Business — Sandbox immédiat DISPONIBLE

Wise propose un environnement sandbox **auto-service, gratuit et sans KYB**. Vous obtenez un compte de test en quelques minutes avec un solde fictif et un Profile ID utilisable pour tester l'intégration complète.

3.1 — Création du compte sandbox

- 1 Aller sur <https://sandbox.transferwise.tech/register>
- 2 Choisir le type **Business** (pas Personal — l'API Business est différente)
- 3 Renseigner email + mot de passe + nom d'entreprise (libre, aucune vérification)
- 4 Confirmer l'email
- 5 Se connecter au dashboard sandbox

3.2 — Récupération des credentials

CREDENTIAL	OÙ LE TROUVER	CHAMP CLENZY CORRESPONDANT
API Token (secret)	Dashboard → <i>Settings</i> → <i>API tokens</i> → bouton "Create new token" avec scope <code>Full access</code>	<code>wise.api-token</code>
Profile ID (numérique)	Dashboard → <i>Account</i> → en haut, sous le nom du business. Format : un long entier (ex: 12345678)	<code>wise.profile-id</code>

Format du token : `11111111-2222-3333-4444-555555555555` (UUID).

3.3 — Configuration du webhook

- 1 Dashboard Wise → *Settings* → *Webhooks*
- 2 Cliquer sur "**Subscribe to a webhook**"
- 3 Renseigner l'URL :
`https://<votre-tunnel-ngrok>/api/webhooks/payouts/wise/transfers-state-change`
- 4 Sélectionner l'événement **transfers#state-change**

5 Wise affiche une **clé publique RSA** — copier le contenu PEM (commence par `-----BEGIN PUBLIC KEY-----`)

6 Sauvegarder

3.4 — Variables d'environnement Clenzy

Dans `server/src/main/resources/application.yml` (ou via variables d'env) :

```
wise:
  api-token: ${WISE_API_TOKEN:11111111-2222-3333-4444-555555555555}
  profile-id: ${WISE_PROFILE_ID:12345678}
  sandbox: true # false en production
  public-key: | # Clé publique RSA pour vérifier les webhooks
    -----BEGIN PUBLIC KEY-----
    MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAYMeDY1aR...
    -----END PUBLIC KEY-----
```

Wise unique pour Clenzy entier, pas par-tenant

Contrairement à PayTabs/CMI (paiements entrants, credentials par-org), Wise est un compte **unique pour toute la plateforme Clenzy**. Toutes les organisations utilisent le même compte Wise Business comme source pour les payouts. La configuration est globale, pas dans la BDD multi-tenant.

3.5 — Solde du compte sandbox

Pour pouvoir effectuer des virements, le compte Wise sandbox doit avoir un solde. Top-up :

- 1 Dashboard Wise → *Balances*
- 2 Sélectionner **EUR** (ou la devise source de Clenzy)
- 3 Cliquer "**Add money**" → choisir *Bank transfer (sandbox)*
- 4 Le sandbox crédite automatiquement (~10 secondes)
- 5 Vérifier le solde EUR : doit être > 0 pour que les virements partent

3.6 — Couverture géographique

Wise sandbox supporte tous les pays de production. Pour Clenzy, devises clés :

DEVISE	PAYS	DÉLAI VIREMENT SANDBOX
EUR → MAD	vers Maroc	Quasi-instantané en sandbox
EUR → SAR	vers Arabie Saoudite	Quasi-instantané en sandbox
EUR → EUR	Zone SEPA	Quasi-instantané en sandbox
EUR → USD	USA	Quasi-instantané en sandbox

4. GoCardless PIS — Sandbox immédiat

DISPONIBLE

EU PSD2

GoCardless Bank Account Data inclut un service Payment Initiation (PIS) qui permet d'initier des virements SEPA Credit Transfer depuis le compte Clenzy bancaire sans uploader de XML pain.001. Le sandbox est gratuit et inclut une **banque virtuelle de test** (`SANDBOXFINANCE`) qui simule le flow complet de SCA.

4.1 — Création du compte sandbox

- 1 Aller sur <https://bankaccountdata.gocardless.com/register/>
- 2 Renseigner email + mot de passe + nom de la société
- 3 Confirmer l'email
- 4 Dashboard sandbox accessible immédiatement

4.2 — Récupération des credentials

CREDENTIAL	OÙ LE TROUVER	CHAMP CLENZY CORRESPONDANT
Secret ID	Dashboard → <i>User secrets</i> → "Generate new secret"	<code>gocardless.secret-id</code>
Secret Key (secret)	Affiché une seule fois lors de la génération — copier immédiatement	<code>gocardless.secret-key</code>

4.3 — Setup initial : authentication de votre compte bancaire Clenzy

Pour pouvoir initier des virements PIS, GoCardless doit avoir un consentement valide pour **votre compte bancaire Clenzy** (le débiteur). C'est un SCA one-time à faire en tant qu'admin.

- 1 Une fois Clenzy démarré, ouvrir Settings → Reversements → Mes reversements (en mode admin/host)
- 2 Cliquer sur "**Choisir une autre méthode**"
- 3 Sélectionner "**Open Banking PIS**"
- 4 Choisir `SANDBOXFINANCE_SF00000` dans le menu déroulant (la banque de test GoCardless)
- 5 Cliquer sur "**Connecter ma banque**"

- 6 Le browser est redirigé vers le portail SCA sandbox de GoCardless
- 7 Choisir un compte fictif (ex: "Current Account")
- 8 Autoriser (l'authentification est simulée — aucun mot de passe requis en sandbox)
- 9 Browser redirigé vers Clenzly avec `?openbanking=callback`
- 10 Le frontend appelle automatiquement `finalizeOpenBankingCallback` → consent activé pour 90 jours

4.4 — Récupération de l'ID du compte débiteur

Une fois le consentement validé, GoCardless vous a fourni un **account_id** qui identifie votre compte bancaire pour les futurs virements. Cet ID est nécessaire dans la config Clenzly.

- 1 Dashboard GoCardless → *Account & transactions* → *Accounts*
- 2 Trouver le compte qu'on vient d'autoriser
- 3 Copier l'**Account ID** (UUID)

4.5 — Configuration du webhook

- 1 Dashboard GoCardless → *Developers* → *Webhook endpoints*
- 2 Cliquer "**Create new endpoint**"
- 3 URL : `https://<votre-tunnel-ngrok>/api/webhooks/payouts/gocardless/events`
- 4 Sélectionner les événements :
 - `payments.paid`
 - `payments.confirmed`
 - `payments.failed`
 - `payments.cancelled`
 - `payments.customer_approval_denied`
- 5 GoCardless génère un **webhook secret** — copier immédiatement

4.6 — Variables d'environnement Clenzzy

```
gocardless:
  secret-id: ${GOCARDLESS_SECRET_ID}
  secret-key: ${GOCARDLESS_SECRET_KEY}
  debtor-account-id: ${GOCARDLESS_DEBTOR_UUID}    # UUID du compte Clenzzy
  webhook-secret: ${GOCARDLESS_WEBHOOK_SECRET}
```

4.7 — Banques disponibles en sandbox

GoCardless expose ~2400 banques européennes en production. En sandbox, on a une banque virtuelle de test :

INSTITUTION ID	NOM	PAYS	USAGE
SANDBOXFINANCE_SFIN0000	Sandbox Finance	FR/EU	Tests généraux, SCA simulé
BNP_PARIBAS_BNPAFRPP	BNP Paribas	FR	Prod — KYB requis
SOCIETE_GENERALE_S0GEFRPP	Société Générale	FR	Prod — KYB requis

Endpoint banques dynamique côté Clenzzy

Le dialog frontend appelle `GET /api/owner-payout-config/openbanking/institutions?country=FR` qui proxy GoCardless. La liste est mise en cache 1h côté backend. En cas d'erreur réseau, fallback sur une liste de 8 banques codées en dur.

5. Configuration Clenzy

5.1 — application.yml complet (extrait Payouts)

```
clenzy:
  base-url: ${CLENZY_BASE_URL:http://localhost:8084} # ou URL ngrok

# Stripe Connect (existant, EU payouts)
stripe:
  secret-key: ${STRIPE_SECRET_KEY:sk_test_xxx}
  connect-platform-id: ${STRIPE_CONNECT_PLATFORM:acct_xxx}

# Wise Business (international payouts)
wise:
  api-token: ${WISE_API_TOKEN:}
  profile-id: ${WISE_PROFILE_ID:}
  sandbox: ${WISE_SANDBOX:true}
  public-key: ${WISE_PUBLIC_KEY:}

# GoCardless PIS (auto-SEPA EU)
gocardless:
  secret-id: ${GOCARDLESS_SECRET_ID:}
  secret-key: ${GOCARDLESS_SECRET_KEY:}
  debtor-account-id: ${GOCARDLESS_DEBTOR_UUID:}
  webhook-secret: ${GOCARDLESS_WEBHOOK_SECRET:}
```

5.2 — Migration Liquibase

La migration `0124__add_wise_and_openbanking_to_owner_payout_config.sql` ajoute 5 colonnes à `owner_payout_config` :

COLONNE	TYPE	USAGE
<code>wise_recipient_id</code>	VARCHAR(64)	ID du recipient Wise créé lors du 1er payout
<code>wise_profile_id</code>	VARCHAR(64)	Optionnel — profile Wise de l'owner
<code>open_banking_provider</code>	VARCHAR(20)	GOCARDLESS ou TINK
<code>open_banking_consent_id</code>	VARCHAR(128)	Requisition ID GoCardless
<code>open_banking_consent_expires_at</code>	TIMESTAMP	Expiration consent (typiquement +90j)

Production : migration via workflow CI/CD

En dev, Liquibase s'applique automatiquement au démarrage Spring Boot. En production, `SPRING_LIQUIBASE_ENABLED=false` et il faut passer par le workflow GitHub Actions *Liquibase Bootstrap* de clenzy-infra avec un `sync_tag`.

5.3 — Configuration côté propriétaire

L'admin ou le propriétaire ouvre Settings → Reversements → Mes reversements → bouton "**Choisir une autre méthode**" :

MÉTHODE	CHAMPS REQUIS
Wise	IBAN + BIC + Titulaire du compte (renseignés directement dans le dialog)
Open Banking	Sélection de la banque dans le dropdown + clic "Connecter ma banque" → flow SCA
SEPA Transfer	IBAN + BIC + Titulaire
Stripe Connect	Géré séparément via onboarding self-service
Manuel	Aucun

6. Scénarios de test end-to-end

6.1 — Scénario Wise complet (EUR → MAD, propriétaire au Maroc)

Préparation : Wise sandbox configuré, solde EUR > 1000€, webhook actif, ngrok actif, propriétaire fictif créé dans Clenzy avec IBAN marocain.

- 1 SUPER_ADMIN ouvre Settings → Reversements → table des propriétaires
- 2 Clic sur l'icône engrenage du propriétaire marocain → modale "Méthode de reversement"
- 3 Sélectionner "**Wise Business**" dans les options
- 4 Renseigner l'IBAN marocain (ex: `MA64011519000001205000534921`) + Titulaire
- 5 Cliquer "**Enregistrer**"
- 6 Générer un payout via le scheduler ou manuellement depuis Facturation → Reversements
- 7 Approuver le payout (status passe à APPROVED)
- 8 Exécuter le payout (clic icône "Exécuter") :
 - Backend : `WisePayoutExecutor.execute()`
 - 1er appel : `POST /v1/accounts` crée le recipient Wise (idempotent côté Wise)
 - 2e appel : `POST /v3/profiles/<id>/quotes` obtient un quote EUR → MAD
 - 3e appel : `POST /v1/transfers` crée le transfer avec idempotence UUID
 - 4e appel : `POST /v3/profiles/<id>/transfers/<tid>/payments` funde depuis le solde
- 9 Payout passe en **PROCESSING** avec `paymentReference = "WISE:<transferId>"`
- 10 Wise traite le virement (quasi-instantané en sandbox)
- 11 Webhook `transfers#state-change` reçu avec `current_state = outgoing_payment_sent`
- 12 Vérification signature RSA-SHA256 avec la clé publique Wise
- 13 Lookup payout par `paymentReference` → mark PAID + notify success
- 14 Dashboard Wise : transfer visible dans l'historique

6.2 — Scénario Open Banking complet (consent + virement)

- 1 Propriétaire (ou admin) ouvre Mes reversements
- 2 Clic "Choisir une autre méthode" → modale
- 3 Sélectionner "**Open Banking PIS**" → dropdown se charge depuis GoCardless
- 4 Choisir `SANDBOXFINANCE_SF0000`
- 5 Clic "**Connecter ma banque**" :
 - Frontend : `POST /api/owner-payout-config/me/openbanking/init`
 - Backend : `GoCardlessPisClient.createRequisition()` → URL SCA + requisitionId stocké en BDD
 - Frontend : `window.location.href = redirectUrl`
- 6 Portail SANDBOXFINANCE : sélection du compte fictif + auth simulée
- 7 Redirect vers Clenzy : `app.clenzy.fr/settings?tab=reversements&openbanking=callback`
- 8 `useEffect` dans `MyPayoutSettings` détecte le query param → appel `finalizeOpenBankingCallback`
- 9 Backend : vérifie le consentement valide chez GoCardless → marque `verified=true` + `payoutMethod=OPEN_BANKING` + `consentExpiresAt = now() + 90 days`
- 10 Snackbar "Méthode de reversement mise à jour"
- 11 Bannière `OpenBankingConsentBanner` affichée avec consentement actif
- 12 Maintenant, à chaque payout APPROVED → `OpenBankingPayoutExecutor.execute()` initie un paiement GoCardless
- 13 Webhook `payments.paid` reçu → mark PAID

6.3 — Tester l'expiration du consentement (forcer date passée)

Pour valider la bannière proactive, on peut forcer une date d'expiration passée en BDD :

```
$ docker exec -it clenzy-postgres-dev psql -U clenzy -d clenzy -c \  
"UPDATE owner_payout_config  
SET open_banking_consent_expires_at = NOW() - INTERVAL '1 day'  
WHERE owner_id = <OWNER_ID>;"
```

Recharger la page Mes reversements → bannière rouge "**Consent bancaire expiré**" avec bouton "Reconnecter ma banque".

Pour tester l'avertissement J-7 :

```
$ UPDATE owner_payout_config
SET open_banking_consent_expires_at = NOW() + INTERVAL '3 days'
WHERE owner_id = <OWNER_ID>;
```

→ bannière jaune **"Consent bancaire à renouveler bientôt"**.

6.4 — Tester un refund Wise

L'API Wise ne supporte pas les refunds directs — un payout envoyé ne peut pas être annulé une fois `outgoing_payment_sent`. Si le transfer est encore en état `incoming_payment_waiting`, on peut le cancel via dashboard.

Pour Clenzy : `WisePayoutExecutor.refundPayment` n'est pas implémenté (hors scope payouts). Si un refund est nécessaire, il faut le faire manuellement via dashboard Wise + appeler `markAsFailed` côté Clenzy.

7. Débogage & commandes utiles

7.1 — Logs backend Clenzy

```
$ tail -f server/logs/application.log | grep -iE "wise|gocardless|payout"
```

Logs clés :

- Wise transfer XXX created+funded for payout YYY : transfer créé et fundé avec succès
- Wise webhook : payout YYY marque PAID (transferId=XXX) : webhook traité
- Wise webhook : signature invalide : vérifier wise.public-key
- GoCardless institutions chargees : N banques pour pays=FR : cache rafraîchi
- Open Banking : consent XXX active pour owner Y (org Z) : SCA finalisé
- Open Banking payment XXX initiated for payout YYY : virement PIS lancé

7.2 — Inspection BDD

```
$ docker exec -it clenzy-postgres-dev psql -U clenzy -d clenzy -c \  
"SELECT id, owner_id, payout_method, wise_recipient_id, open_banking_provider,\  
    open_banking_consent_id, open_banking_consent_expires_at, verified\  
FROM owner_payout_config\  
ORDER BY updated_at DESC LIMIT 10;"
```

```
$ docker exec -it clenzy-postgres-dev psql -U clenzy -d clenzy -c \  
"SELECT id, owner_id, status, payout_method, net_amount, currency,\  
    payment_reference, paid_at, failure_reason\  
FROM owner_payouts\  
ORDER BY created_at DESC LIMIT 20;"
```

7.3 — Dashboard ngrok

Ouvrir <http://127.0.0.1:4040> pendant que ngrok tourne pour voir les webhooks entrants en temps réel. Particulièrement utile pour debugger les signatures Wise (RSA) et HMAC GoCardless.

7.4 — Replay d'un webhook Wise depuis le dashboard

Dashboard Wise → Webhooks → cliquer sur l'événement → bouton **"Replay"**. Permet de re-déclencher un webhook sans refaire un transfert complet.

7.5 — Verifier un consentement GoCardless

Via curl direct (utile si on doute de la validité côté GoCardless) :

```
$ # 1. Obtenir un token
TOKEN=$(curl -s -X POST https://bankaccountdata.gocardless.com/api/v2/token/new/ \
  -H "Content-Type: application/json" \
  -d '{"secret_id":"'GOCARDLESS_SECRET_ID',"secret_key":"'GOCARDLESS_SECRET_KEY'"}' \
  | jq -r .access)

# 2. Lookup une requisition
curl -s https://bankaccountdata.gocardless.com/api/v2/requisitions/<REQ_ID>/ \
  -H "Authorization: Bearer $TOKEN" | jq .

# Status attendu : "LN" (Linked) pour un consentement valide
```

7.6 — Tests unitaires backend

```
$ cd server && mvn test -
Dtest='WisePayoutExecutorTest,OpenBankingPayoutExecutorTest,PayoutExecutorRegistryTest'
```

8. Erreurs courantes & solutions

SYMPTÔME	CAUSE PROBABLE	SOLUTION
"Wise n'est pas configure cote Clenzy"	Variables <code>wise.api-token</code> ou <code>wise.profile-id</code> vides	Vérifier <code>application.yml</code> + redémarrer le backend
Wise renvoie 401 sur createQuote	API token invalide ou expiré	Régénérer un nouveau token dans dashboard Wise → Settings → API tokens
Wise renvoie "insufficient balance"	Solde EUR sandbox insuffisant	Top-up via dashboard Wise → Balances → Add money
Wise webhook 401 "Invalid signature"	Clé publique RSA mal copiée ou changée	Re-copier la clé publique depuis dashboard Wise → Settings → Webhooks
Wise transfer reste en PROCESSING	Webhook non reçu (ngrok down ou URL mal config)	Vérifier ngrok actif + URL dans dashboard Wise correspond à ngrok actuel
"Open Banking : aucun consent SCA enregistré"	Premier setup : aucune requisition encore créée	L'admin doit ouvrir "Connecter ma banque" depuis Mes reversements
"Open Banking : le consent SCA est expire"	Plus de 90 jours depuis le dernier SCA	Refaire le SCA via la bannière rouge "Reconnecter ma banque"
GoCardless renvoie 403 sur createRequisition	Secret ID/Key invalides ou expirés	Régénérer dans dashboard → User secrets
Liste banques GoCardless ne charge pas dans le dialog	Erreur réseau ou API down	Le fallback codé en dur prend le relais. Vérifier console réseau frontend.
Webhook GoCardless 401 "Invalid signature"	Webhook secret mal copié	Régénérer un webhook endpoint dans dashboard + copier le nouveau secret
ngrok change d'URL à chaque relance	Plan gratuit ngrok	Mettre à jour les URLs webhook dans Wise + GoCardless, ou passer au plan payant

9. Annexes — webhooks & structure des payloads

9.1 — Webhook Wise (transfers#state-change)

URL Clenzy : `POST /api/webhooks/payouts/wise/transfers-state-change`

Header signature : `X-Signature-SHA256: <base64-encoded RSA signature>`

Body (extrait) :

```
{
  "data": {
    "resource": {
      "id": 123456789,
      "type": "transfer",
      "profile_id": 12345678,
      "account_id": 87654321
    },
    "current_state": "outgoing_payment_sent",
    "previous_state": "processing",
    "occurred_at": "2026-05-23T10:42:00Z"
  },
  "subscription_id": "subscription-uuid",
  "event_type": "transfers#state-change",
  "schema_version": "2.0.0",
  "sent_at": "2026-05-23T10:42:01Z"
}
```

États possibles :

- `processing` : transfer reçu côté Wise, en attente d'exécution
- `funds_converted` : conversion devise effectuée
- `outgoing_payment_sent` : virement émis → Clenzy mark PAID
- `funds_refunded`, `cancelled`, `charged_back`, `bounced_back` → Clenzy mark FAILED

9.2 — Webhook GoCardless (payments.*)

URL Clenzy : `POST /api/webhooks/payouts/gocardless/events`

Header signature : `Webhook-Signature: <hex HMAC-SHA256>`

Body (extrait) :

```
{
  "events": [
    {
      "id": "EV01XXX",
      "created_at": "2026-05-23T10:42:00Z",
      "resource_type": "payments",
      "action": "paid",
      "links": {
        "payment": "PM01XXX",
        "creditor": "CR01XXX"
      },
      "details": {
        "origin": "bank",
        "cause": "payment_paid",
        "scheme": "sepa_credit_transfer"
      }
    }
  ]
}
```

Actions traitées par Clenzy :

- `paid`, `confirmed` : succès → mark PAID
- `failed`, `cancelled`, `customer_approval_denied` : échec → mark FAILED

9.3 — Vérification de signature Wise (RSA-SHA256)

```
Signature signer = Signature.getInstance("SHA256withRSA");
signer.initVerify(wisePublicKey);
signer.update(rawPayloadBody.getBytes(StandardCharsets.UTF_8));
byte[] receivedSig = Base64.getDecoder().decode(headerSignature);
boolean valid = signer.verify(receivedSig);
```

9.4 — Vérification de signature GoCardless (HMAC-SHA256)

```
Mac mac = Mac.getInstance("HmacSHA256");
mac.init(new SecretKeySpec(webhookSecret.getBytes(UTF_8), "HmacSHA256"));
byte[] computed = mac.doFinal(rawPayloadBody.getBytes(UTF_8));
String computedHex = HexFormat.of().formatHex(computed);
boolean valid = constantTimeEquals(computedHex, headerSignature);
```

9.5 — Endpoints Clenzy exposés

MÉTHODE	ENDPOINT	USAGE
POST	<code>/api/owner-payout-config/me/openbanking/init</code>	Self-service init SCA
POST	<code>/api/owner-payout-config/{ownerId}/openbanking/init</code>	Admin init SCA pour un owner
GET	<code>/api/owner-payout-config/openbanking/callback</code>	Finalize après retour SCA
GET	<code>/api/owner-payout-config/openbanking/institutions?country=FR</code>	Liste banques GoCardless (cache 1h)
POST	<code>/api/webhooks/payouts/wise/transfers-state-change</code>	Webhook Wise
POST	<code>/api/webhooks/payouts/gocardless/events</code>	Webhook GoCardless

Vous voilà prêt

Avec le sandbox Wise gratuit et le sandbox GoCardless avec SANDBOXFINANCE, vous pouvez tester l'intégralité du flow payouts P2 sans accès à du KYB réel. Les credentials de production demanderont juste 2-3 semaines d'onboarding côté Wise + une vérification GoCardless. Le code Clenzy est déjà prêt pour la production : seules les variables d'env changent.